LAB-2752: Build Reusable Agentic AI Patterns for Cold Chain: Predictive & Autonomous Workflows

Hands-on lab guide

Shanmugaraja Selvam
shanmsel@in.ibm.com
Solution Architect, IBM Build Engineering

Niteesh Nair
niteesh.nair@ibm.com
AI Engineer, IBM Build Engineering

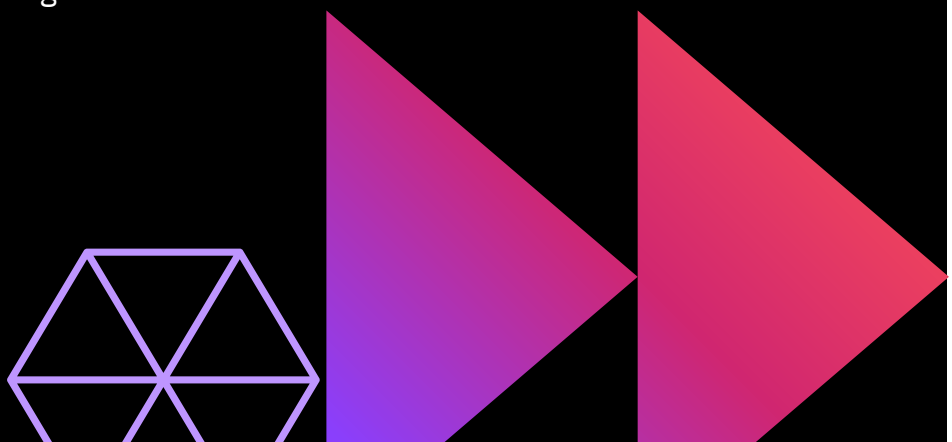


Table of Contents

1 Introduction 5

2 About this hands-on lab 5

3 Did not complete the lab...?? (for Students) 6

4 Phase 1: Explore the Fleet Backend 6

4.1 What is Already Built 7
4.2 Verify the APIs 8
4.3 Understand TRUCK-003 9
4.4 Run the Pipeline in Mock Mode 9

5 Phase 2: Build the Forecast Service 10

5.1 Install the TTM Forecasting Mode 10
5.2 Build the Forecast Service with Bob 10
5.3 Verify the Forecast Service 11

6 Phase 3: Build the Agents and Tools 12

7 Phase 4: Build the Dashboard and Run the Live Pipeline 14

7.1 Build the Dashboard 14
7.2 Verify the Dashboard 14
7.3 Run the Live Pipeline 16

8 Getting Help and Troubleshooting 17

1. Reach out to the Lab instructor. 17
2. Common troubleshooting tips 17
3. Getting help 17

9 Lab Reference Materials 17

Notices and disclaimers

© 2026 International Business Machines Corporation. No part of this document may be reproduced or transmitted in any form without written permission from IBM.

U.S. Government Users Restricted Rights — use, duplication or disclosure restricted by GSA ADP Schedule Contract with IBM.

This document is current as of the initial date of publication and may be changed by IBM at any time. Not all offerings are available in every country in which IBM operates.

Information in these presentations (including information relating to products that have not yet been announced by IBM) has been reviewed for accuracy as of the date of initial publication and could include unintentional technical or typographical errors. IBM shall have no responsibility to update this information.

This document is distributed “as is” without any warranty, either express or implied. In no event, shall IBM be liable for any damage arising from the use of this information, including but not limited to, loss of data, business interruption, loss of profit or loss of opportunity. IBM products and services are warranted per the terms and conditions of the agreements under which they are provided. The performance data and client examples cited are presented for illustrative purposes only. Actual performance results may vary depending on specific configurations and operating conditions.

IBM products are manufactured from new parts or new and used parts.
In some cases, a product may not be new and may have been previously installed. Regardless, our warranty terms apply.”

Any statements regarding IBM's future direction, intent or product plans are subject to change or withdrawal without notice.

Performance data contained herein was generally obtained in a controlled,
isolated environments. Customer examples are presented as illustrations of how those customers have used IBM products and the results they may have achieved. Actual performance, cost, savings or other results in other operating environments may vary.

References in this document to IBM products, programs, or services does not imply that IBM intends to make such products, programs or services available in all countries in which IBM operates or does business.

Workshops, sessions and associated materials may have been prepared by independent session speakers, and do not necessarily reflect the views of IBM. All materials and discussions are provided for informational purposes only, and are neither intended to, nor shall constitute legal or other guidance or advice to any individual participant or their specific situation.

It is the customer’s responsibility to ensure its own compliance with legal requirements and to obtain advice of competent legal counsel as to the identification and interpretation of any relevant laws and regulatory requirements that may affect the customer’s business and any actions the customer may need to take to comply

with such laws. IBM does not provide legal advice or represent or warrant that its services or products will ensure that the customer follows any law.

Notices and disclaimers (Continued)

Questions on the capabilities of non-IBM products should be addressed to the suppliers of those products. IBM does not warrant the quality of any third-party products, or the ability of any such third-party products to interoperate with IBM's products. IBM expressly disclaims all warranties, expressed or implied, including but not limited to, the implied warranties of merchantability and fitness for a purpose.

The provision of the information contained herein is not intended to, and does not, grant any right or license under any IBM patents, copyrights, trademarks or other intellectual property right.

IBM, the IBM logo, and ibm.com are trademarks of International Business Machines Corporation, registered in many jurisdictions worldwide. Other product and service names might be trademarks of IBM or other companies. A current list of IBM trademarks is available on the Web at "Copyright and trademark information" at: www.ibm.com/legal/copytrade.shtml.

1 Introduction

Cold-chain logistics is no longer just a transportation problem — it is an operational risk management challenge where timing is everything. When a temperature breach, a storm, and a cooling system fault converge on the same truck, an operator has minutes to act. In this workshop you will build the intelligence layer of a cold-chain logistics platform using IBM Bob and IBM AI building blocks.

The fleet backend is already built and running in your lab environment. Your job is to build three things on top of it: a forecasting service that predicts temperature breach risk using IBM Granite TTM-R2, a five-agent watsonx Orchestrate pipeline that autonomously decides and acts on that risk, and a live dashboard that lets you trigger and track everything in real time.

You will use two IBM Bob build modes that are bundled in the lab repository under bob-modes-skills/ — install them in Lab 0 before starting:

TTM Forecasting Mode — guides you through building the forecast service with IBM Granite TTM-R2
FleetOps Agent Builder Mode — guides you through building the five watsonx Orchestrate agents and their Python tools

These modes know the cold-chain domain, the agent orchestration patterns, and the exact API contracts. Let Bob guide you — answer its questions, review the generated code, apply it, and move forward.

2 About this hands-on lab

By the end of this 90-minute lab you will be able to:

1. Configure and start the pre-built FleetOps backend and verify its REST APIs via Swagger UI
2. Build a FastAPI temperature-forecasting service using IBM Granite TTM-R2 with IBM Bob
3. Define five watsonx Orchestrate agents (weather · station · route · decision · notification) and upload their Python tools
4. Register watsonx Orchestrate connections and wire all five agent IDs to the fleet backend
5. Complete a Carbon-based live dashboard using IBM Bob in Agent mode
6. Run the end-to-end agentic pipeline on TRUCK-003 and observe the autonomous reroute decision

3 Did not complete the lab...?? (for Students)

If you did not finish during the session, you can continue in the self-paced lab room using the same environment. Ask your instructor for the self-paced room access details before leaving.

4 Phase 1: Explore the Fleet Backend

The workshop has four phases:

Phase 1 — Explore the Fleet Backend: The backend is pre-built and running. Learn what it exposes, verify the APIs, and understand TRUCK-003 — the critical incident truck you will build intelligence around.

Phase 2 — Build the Forecast Service: Use the TTM Forecasting Bob mode to build a FastAPI forecasting service powered by IBM Granite TTM-R2. It will predict temperature breach risk for TRUCK-003 over a 24-hour horizon.

Phase 3 — Build the Agents and Tools: Use the FleetOps Agent Builder Bob mode to build five watsonx Orchestrate agents and their Python tools. Upload them to watsonx Orchestrate and record the agent IDs.

Phase 4 — Build the Dashboard and Run the Live Pipeline: Wire the agent IDs to the backend, build the Carbon dashboard, and run the full end-to-end agentic workflow on TRUCK-003 — watching the route change live.

Suggested time budget: Phase 1 — 10 min | Phase 2 — 20 min | Phase 3 — 30 min | Phase 4 — 15 min | Buffer — 5 min

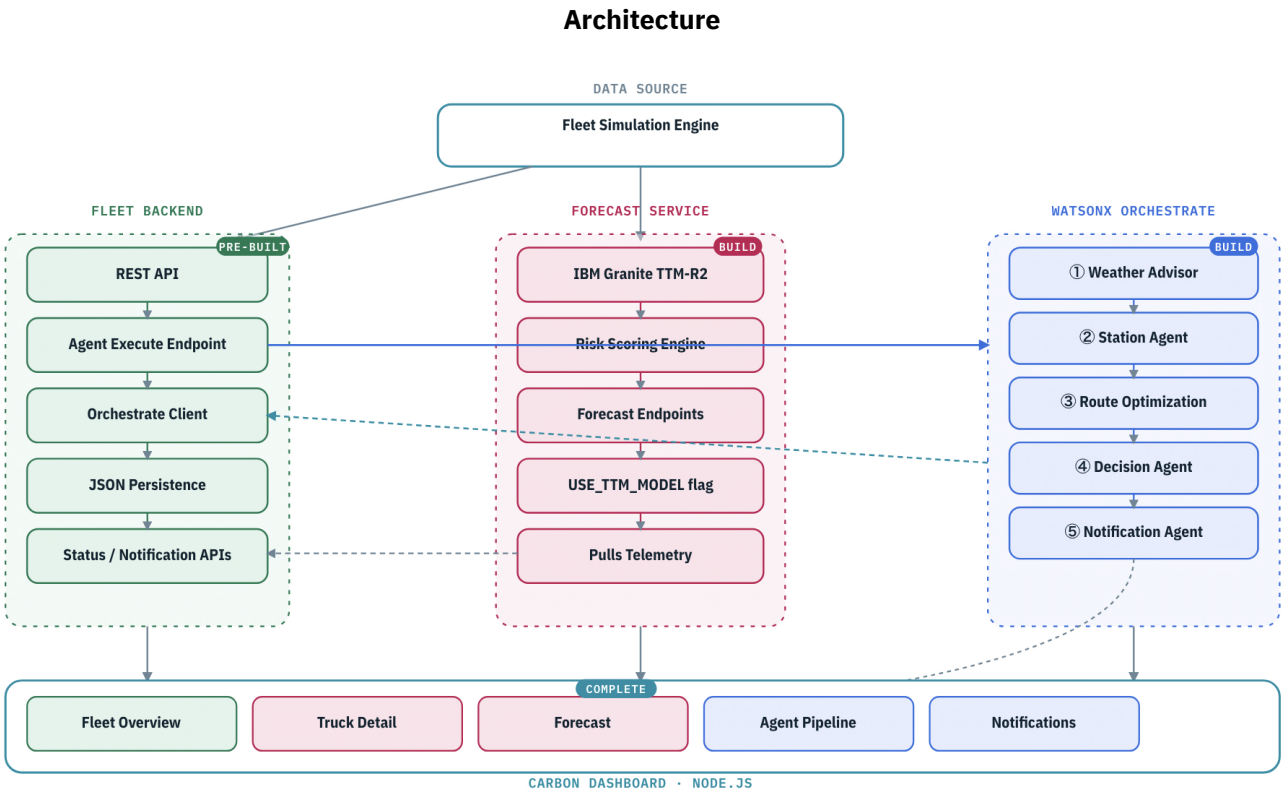
⚡ Checkpoint: If Phase 2 fails (pip install errors or missing WML credentials), ask IBM Bob (TTM Forecasting mode) to regenerate the forecast service and continue to Phase 3.

Acronyms: API — Application Programming Interface | TTM — Tiny Time Mixer (IBM Granite forecasting model) | IAM — IBM Identity and Access Management | ADK — Agent Development Kit (watsonx Orchestrate)

Prerequisites: IBM Bob available in your lab environment with the lab-2752-reusable-agentic-patterns workspace open | Python 3.12+ | Node.js 18+ | watsonx Orchestrate SaaS access — credentials provided by instructor | Lab repository cloned from https://github.ibm.com/shanmsel/lab-2752-reusable-agentic-patterns

git clone https://github.ibm.com/shanmsel/lab-2752-reusable-agentic-patterns

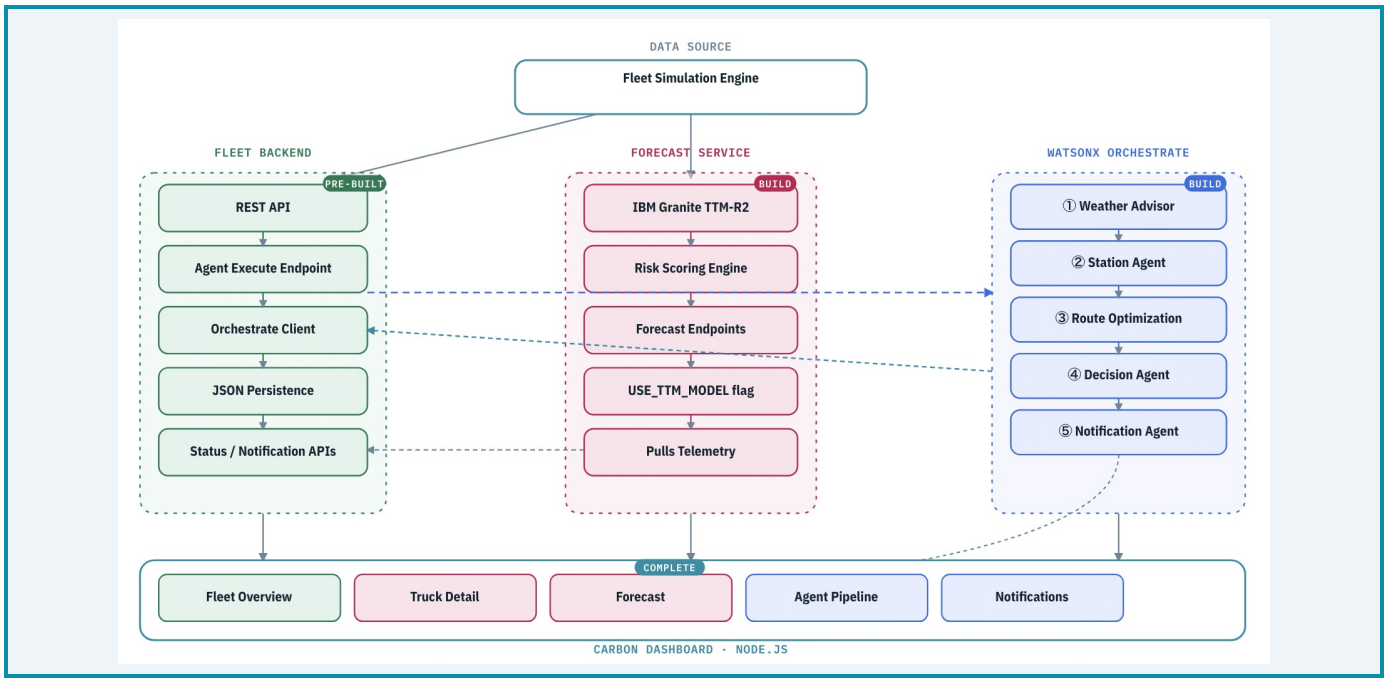
4.1 What is Already Built



The fleet backend (01-fleet-backend) is fully built and running at http://localhost:8085. It provides:

- GET /api/trucks — returns live telemetry for 10 simulated cold-chain trucks
- GET /api/trucks/{truck_id} — single truck detail with temperature, location, cargo, route
- GET /api/alerts — active temperature and weather alerts
- POST /api/agents/execute — triggers the five-agent watsonx Orchestrate pipeline for a truck
- GET /api/agents/status/{execution_id} — polls the pipeline execution status
- GET /api/agents/notifications/{truck_id} — retrieves notifications sent by the pipeline
- GET /api/agents/history — historical pipeline executions

The simulation engine runs a background loop every 5 seconds, advancing truck positions and drifting temperatures. Four trucks have pre-assigned incidents. TRUCK-003 is your critical scenario truck.



Architecture overview — three services: Fleet Backend (:8085, pre-built) · Forecast Service (:5001, you build) · watsonx Orchestrate Agents (you build) · Carbon Dashboard (:4000, you complete). See lab-2752-architecture.html.

4.2 Verify the APIs

First, start the fleet backend (if not already running):
cd 01-fleet-backend
python3.12 -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --host 127.0.0.1 --port 8085 --reload.

Open http://localhost:8085/docs in your browser. This is the Swagger UI for the fleet backend.

1. Expand GET /api/trucks. Click Try it out → Execute. You should receive a list of 10 trucks. Each has a truckId, telemetry (temperature, coolantStatus, currentLocation, speed), cargo (type, value, criticalThreshold), currentTrip (origin, destination, plannedRoute, estimatedArrival), and status.

2. Expand GET /api/alerts. Execute. Note any active alerts — look for type TEMPERATURE_BREACH and severity CRITICAL.

3. Ask IBM Bob (in Agent mode): 'Explain the fleet backend API structure — what does each endpoint return and how do the trucks, alerts, and agents endpoints connect to each other?'
Review the explanation before continuing.

4.3 Understand TRUCK-003

TRUCK-003 is your incident truck for this workshop. Call GET /api/trucks/TRUCK-003 in the Swagger UI.

Key values to note:
- temperature: ~9.2°C and rising (threshold is 8.0°C — already in breach)
- coolantStatus: NORMAL (this is a WEATHER incident, not a coolant failure)
- cargo type: pharmaceuticals, cargo value: ~€180,000
- currentTrip: Atlanta, GA → Macon, GA on I-75, ~100 km remaining
- estimatedArrival: approximately 1.5 hours from now
- incidentType: WEATHER_FAILURE (severe weather system directly on route)

This is the scenario: the cargo is already above threshold, a severe weather system sits directly on the I-75 corridor, and there is under 2 hours to act before the pharmaceutical cargo is unrecoverable. The agents you will build in Phase 3 will analyse this situation and recommend an emergency reroute to the nearest cold storage facility.

4.4 Run the Pipeline in Mock Mode

Before building the real agents, run the pipeline in mock mode to understand the full flow.

1. In the Swagger UI, expand POST /api/agents/execute. Click Try it out. Enter the request body:
   { "truck_id": "TRUCK-003" }
   Click Execute. You will receive an execution_id and status RUNNING.

2. Copy the execution_id. Expand GET /api/agents/status/{execution_id}. Enter the execution_id. Execute. You will see the five agents — weather, station, route, decision, notification — each with a status field. Poll this endpoint every 5 seconds until status is COMPLETED.

3. When COMPLETED, read the decision agent output. In mock mode it returns:
   decision: EMERGENCY_REROUTE, urgency: CRITICAL, riskScore: 85
   recommendation: Reroute to nearest cold storage facility

4. Call GET /api/trucks/TRUCK-003 again. The status field should now show DIVERTED. The simulation engine has applied the mock decision to the truck.

5. Call GET /api/agents/notifications/TRUCK-003. You should see the mock notification stored for the driver.

   This is the exact same flow you will run in Phase 4 — but with real agents making intelligent decisions based on actual weather, station, and route data.

Phase 1 complete.

5 Phase 2: Build the Forecast Service

Phase 2 builds the forecast service using IBM Bob's TTM Forecasting mode. The mode is bundled under bob-modes-skills/ttm-forecast/ in the lab repository — install it now (Lab 0 Step 4a) — it knows how to scaffold a complete FastAPI service with IBM Granite TTM-R2 integration, risk scoring, and all the endpoints the dashboard will need.

5.1 Install the TTM Forecasting Mode

The TTM Forecasting mode files are at bob-modes-skills/ttm-forecast/. You need to install the mode into IBM Bob before using it

a. In your terminal, copy the mode config to the Bob workspace configuration:

```
mkdir -p .bob
cp bob-modes-skills/ttm-forecast/custom_modes.yaml .bob/custom_modes.yaml
cp -r bob-modes-skills/ttm-forecast/skills .bob/skills
```

b. In IBM Bob, click the mode selector (bottom left). You should now see 🔮 TTM Forecasting in the list. Select it.

c. Bob will confirm: 'You are now in TTM Forecasting mode. I specialise in building production-ready time-series forecasting services using IBM Granite TTM models.'

Note: The mode restricts file edits to .py, .yaml, .yml, .md, .txt, .json, .env, and Dockerfile — exactly what you need for this phase.

5.2 Build the Forecast Service with Bob

With the TTM Forecasting mode active, type this prompt into IBM Bob:

I need to build a cold-chain temperature forecast service for a fleet management platform. The service should predict temperature breach risk for pharmaceutical cargo trucks using IBM Granite TTM-R2. It will connect to the fleet backend at http://localhost:8085 to get truck telemetry. The critical temperature threshold is 8.0°C. I need the following API endpoints:
- GET /api/forecast/temperature/{truck_id} — predict breach risk over 24 hours
- GET /api/forecast/station — station availability forecast
- GET /api/forecast/weather/{truck_id} — weather impact forecast
- GET /api/forecast/fleet — fleet-wide optimisation summary
Build the complete service in a folder called 02-forecast-service/.

IBM Bob will ask you 2–3 discovery questions about your data and UI requirements. Answer as follows:
- Data source: The service should fetch truck telemetry from the fleet backend API at http://localhost:8085/api/trucks/{truck_id} and generate synthetic 512-point temperature history based on current temperature.
- UI: No dashboard UI needed — API only. The main dashboard is built separately in Phase 4.

- Model: Use IBM Granite TTM-R2 with USE_TTM_MODEL=False (mock/local fallback — no model download, instant startup. Set True only if you have WML credentials and want the real Granite TTM-R2 model) as the default, with local model as fallback.

Bob will then present a checklist plan. Review it, confirm, and let Bob build the complete service. Apply all generated files to 02-forecast-service/.

5.3 Verify the Forecast Service

After Bob has generated and you have applied all the 02-forecast-service files:

1. Fill in your credentials. Open Open 02-forecast-service/.env (created by Bob). Set:

```
FLEETOPS_API_BASE_URL=http://localhost:8085
USE_TTM_MODEL=False   # WML creds not needed when False
# WML_SPACE_ID=<provided by instructor — only if USE_TTM_MODEL=True>
# WML_API_KEY=<provided by instructor — only if USE_TTM_MODEL=True>
# IBM_CLOUD_REGION=us-south
```

2. Install dependencies and start the service:

```
cd 02-forecast-service
python3.12 -m venv venv
source venv/bin/activate     # Windows: venv\Scripts\activate
pip install -r requirements.txt
uvicorn main:app --host 127.0.0.1 --port 5001 --reload
```

3. Open http://localhost:5001/docs.

4. Call GET /api/forecast/temperature/TRUCK-003. Confirm the response contains:
   - truck_id: TRUCK-003
   - breach_predicted: true
   - risk_level: CRITICAL or HIGH
   - recommendations: list of action strings

5. Call GET /api/forecast/fleet. Confirm trucks_at_risk includes TRUCK-003.

If anything is missing or incorrect, ask IBM Bob (still in TTM mode): 'The forecast endpoint for TRUCK-003 is not returning breach_predicted — help me debug this.' Bob will read the files and fix the issue.

Phase 2 complete.

6 Phase 3: Build the Agents and Tools

Phase 3 builds the five watsonx Orchestrate agents and their Python tools using IBM Bob's FleetOps Agent Builder mode. You will then upload everything to watsonx Orchestrate and collect the five agent IDs needed to wire the live pipeline.

1. STEP 1 — INSTALL THE FLEETOPS AGENT BUILDER MODE

   Run from the repo root:

   ```
   cat bob-modes-skills/agent-modes/custom_modes.yaml >> .bob/custom_modes.yaml
   cp -r bob-modes-skills/agent-modes/rules-fleetops-agent-builder .bob/rules-fleetops-agent-builder
   ```

   Restart Bob (Cmd+Shift+P → Bob: Restart). Select 🚚 FleetOps Agent Builder in the mode selector.

   STEP 2 — GENERATE ALL FIVE AGENTS AND TOOLS WITH BOB

   Send Bob this prompt:

   ```
   Build the complete five-agent pipeline for a cold-chain logistics emergency rerouting workflow.
   The agents run sequentially:
    weather_advisor → station_agent → RouteOptimizationAgent → decision_agent → notification_agent
   The fleet backend API is at http://localhost:8085. Build all five agent YAML files and
   five Python tools with @tool decorators, Pydantic models, and API integration.
   Output everything into agents/ and tools/ folders.
   ```

   Answer Bob's discovery questions:

   ```
   Backend URL:          http://localhost:8085
   Cargo type:           pharmaceutical (2–8°C)
   WhatsApp notification: mock mode — no real Twilio credentials needed
   Plan file:            Yes
   ```

   Bob generates — apply all files:

   ```
   agents/weather_advisor.yaml         tools/weather_forecast/weather_forecast_tool.py
   agents/station_agent.yaml           tools/station_tool/station_search_tool.py
   agents/RouteOptimizationAgent.yaml  tools/route_tool/route_optimizer_tool.py
   agents/decision_agent.yaml          tools/decision_tool/decision_analysis_tool.py
   agents/notification_agent.yaml      tools/notification_tool/whatsapp_notification_tool.py
   ```

2. STEP 3 — UPLOAD AGENTS AND TOOLS TO WATSONX ORCHESTRATE

   Open the watsonx Orchestrate console. Log in.

   For EACH of the five agents:

   a. Build → Agents → New agent → Import from YAML → paste the agent YAML
   b. Copy the Agent ID from the URL bar → record it below
   c. Build → Tools → Upload Python file → upload the tool file
   d. Select the correct connection (see table) and attach the tool to the agent

CONNECTION TABLE — tools silently return empty data without this:

Agent	Tool	Connection
weather_advisor	weather_forecast_tool.py	fleetops_api
station_agent	station_search_tool.py	fleetops_api
RouteOptimizationAgent	route_optimizer_tool.py	(none)
decision_agent	decision_analysis_tool.py	(none)
notification_agent	whatsapp_notification_tool.py	forecast_api

Create connections first (run once):

```
# fleetops_api — used by weather and station tools
orchestrate connections set-credentials -a fleetops_api --env draft \
  -e "FLEETOPS_API_URL=http://localhost:8085" \
  -e "FLEETOPS_API_BASE_URL=http://localhost:8085"

# forecast_api — used by notification tool only
orchestrate connections set-credentials -a forecast_api --env draft \
  -e "FORECAST_API_URL=http://localhost:5001"
```

⚠️ Route and Decision tools need NO connection — they run pure local computation.
⚠️ No real Twilio credentials needed — notification runs in mock mode automatically.

Record Agent IDs:

```
weather_advisor Agent ID:      _____________________________
station_agent Agent ID:        _____________________________
RouteOptimizationAgent ID:     _____________________________
decision_agent Agent ID:       _____________________________
notification_agent Agent ID:   _____________________________
```

Test each agent in the Orchestrate Preview panel before continuing.
Ask Bob: 'What test input should I use to verify the weather_advisor agent?'

STEP 4 — WIRE AGENT IDs TO THE FLEET BACKEND

Open 01-fleet-backend/.env and fill in all five IDs:

```
WATSONX_ORCHESTRATE_URL=https://<your-tenant>.ai.ibm.com
WATSONX_ORCHESTRATE_API_KEY=<your-ibm-cloud-api-key>
WATSONX_ORCHESTRATE_AGENT_WEATHER=<weather_advisor Agent ID>
WATSONX_ORCHESTRATE_AGENT_STATION=<station_agent Agent ID>
WATSONX_ORCHESTRATE_AGENT_ROUTE=<RouteOptimizationAgent ID>
WATSONX_ORCHESTRATE_AGENT_DECISION=<decision_agent Agent ID>
WATSONX_ORCHESTRATE_AGENT_NOTIFICATION=<notification_agent Agent ID>
```

Restart the fleet backend:

```
cd 01-fleet-backend && source venv/bin/activate
uvicorn app.main:app --host 127.0.0.1 --port 8085 --reload
```

✅ Startup log should show: 'WatsonX Orchestrate configuration validated'

Phase 3 complete.

Phase 4 wires everything together. You will build the Carbon dashboard, point it at the fleet backend and forecast service, and run the full end-to-end agentic pipeline on TRUCK-003 in live mode — watching the route change in real time.

7 Phase 4: Build the Dashboard and Run the Live Pipeline

7.1 Build the Dashboard

Switch IBM Bob back to **Agent mode** (default). Send:

"I have a Carbon dashboard at 04-carbon-dashboard/public/.

It has five tabs: Fleet Overview, Truck Detail, Forecast, Agent Pipeline, and Notifications.

The fleet backend runs at http://localhost:8085 and the forecast service at http://localhost:5001.

The Node.js proxy in server.js forwards /api/* to port 8085 and /api/forecast/* to port 5001

Wire up all five tabs so each one fetches live data from the correct endpoint and

updates every 5 seconds. Use vanilla JavaScript with the fetch API — no React needed."

Bob will generate or patch the JavaScript files in 04-carbon-dashboard/public/. Apply all changes.

7.2 Verify the Dashboard

1. Start the Dashboard

   cd 04-carbon-dashboard

   npm install

   node server.js

   Open http://localhost:4000 in your browser.

2. Verify Fleet Overview tab

   The Fleet Overview tab should populate with all 10 trucks within 5 seconds.
   Each truck card shows: truck ID, temperature, status, cargo type.

3. Verify the truck detail tab

   Click the **Truck Detail** tab. Confirm you see the incident truck with:

   - Temperature: above the 8.0 °C threshold (shown with a red indicator)
   - Coolant Status: NORMAL
   - Cargo: pharmaceuticals
   - Route: Atlanta, GA → Macon, GA · I-75
   - Incident: WEATHER_FAILURE

4. Verify Forecast tab

   Click the **Forecast** tab. Confirm:

   - breach_predicted: true
   - risk_level: CRITICAL or HIGH
   - Temperature chart showing the predicted rise

   If any tab shows an error or empty data, use these Bob debug prompts:

   Forecast tab not loading:

   The Forecast tab is not loading data — the fetch to /forecast/api/forecast/temperature/{truck_id}

   is failing. Debug and fix the proxy configuration in server.js.

   Truck Detail tab not showing incident truck:

   The Truck Detail tab is showing all trucks instead of filtering to the incident truck.

   Fix the JavaScript in public/app.js to filter for the truck with incidentType WEATHER_FAILURE.

   Agent Pipeline tab not updating:

   The Agent Pipeline tab is not polling for status updates. The execution_id is returned

   but the status never changes from RUNNING. Fix the polling logic in public/agents.js

   to call GET /api/agents/status/{execution_id} every 3 seconds until COMPLETED.

   Fleet Overview not populating:

   The Fleet Overview tab shows 0 trucks. The fetch to /api/trucks is failing.

   Debug the proxy in server.js — confirm /api/* forwards to port 8085.

   Apply any fixes and refresh.

7.3 Run the Live Pipeline

1. Trigger the Pipeline

   Click the **Agent Pipeline** tab on the dashboard. Find the incident truck and click **Run Pipeline**.

   The dashboard calls POST /api/agents/execute and begins polling

   GET /api/agents/status/{execution_id} every 3 seconds.

2. Click the **Agent Pipeline** tab on the dashboard. Find the incident truck and click **Run Pipeline**.

   The dashboard calls POST /api/agents/execute and begins polling

   GET /api/agents/status/{execution_id} every 3 seconds.

3. Watch the pipeline progress

   You should see five stages light up sequentially:

   Weather Advisor -> Station Agent -> Route Agent -> Decision Agent -> Notification Agent

4. Read the live decision

   When status reaches COMPLETED:

decision:	EMERGENCY_REROUTE
urgency:	CRITICAL
recommended_route:	Nearest cold storage diversion off I-75
confidence:	0.94

5. Confirm the truck rerouted

   Switch to the **Fleet Overview** tab. The incident truck status should now show **DIVERTED**.

6. Check the notification

   You will see notification related details.

8 Getting Help and Troubleshooting

1. Reach out to the Lab instructor.

2. Common troubleshooting tips

Fleet backend not starting: confirm Python 3.12+ with python3 --version. Activate venv and re-run pip install -r requirements.txt. Check the terminal for import errors — these usually mean a model file has a wrong relative import path.

Forecast service slow on first start: pip install takes 3–5 minutes for torch and transformers. If USE_TTM_MODEL=False the model does not download — startup should be under 5 seconds.

watsonx Orchestrate agent failing: in the startup log look for WatsonX Orchestrate configuration validated. If you see a missing env var error, check 01-fleet-backend/.env has all five agent IDs and the correct URL and API key.

Agent pipeline stuck on RUNNING: in mock mode (no WATSONX_ORCHESTRATE_URL set) the pipeline completes in 2–3 seconds. In live mode allow 30 seconds. If it is stuck after 60 seconds, check logs for IAM token errors — your API key may be invalid.

Notification tab empty after pipeline: the forecast backend runs WhatsApp in mock mode when TWILIO_AUTH_TOKEN is absent. Call GET http://localhost:5001/api/notifications/whatsapp/history to confirm the notification was stored. The dashboard Notifications tab reads this endpoint.

Dashboard not showing trucks: confirm the fleet backend is running on port 8085 and the forecast service is running on port 5001. The proxy in server.js forwards requests to these ports.

3. Getting help

For self-paced work: the IBM Bob prompts in this guide are complete and self-contained — you can run them again to regenerate any file. If a generated file has an import error, ask IBM Bob: Explain this error and fix the import in this file. For additional help visit community.ibm.com/techxchange.

9 Lab Reference Materials

IBM Bob (watsonx Code Assistant): https://www.ibm.com/products/watsonx-code-assistant
IBM Granite TTM-R2 model: https://huggingface.co/ibm-granite/granite-timeseries-ttm-r2
watsonx Orchestrate documentation: https://www.ibm.com/docs/en/watsonx/watson-orchestrate
IBM Carbon Design System: https://carbondesignsystem.com
FastAPI documentation: https://fastapi.tiangolo.com

Lab reference repository: https://github.ibm.com/shanmsel/lab-2752-reusable-agentic-patterns
IBM TechXchange 2026 lab download site: provided by your instructor.

Folder structure:
00-setup/ — credentials, Bob modes, env files
01-fleet-backend/ — pre-built FastAPI backend (pre-built, read-only)
02-forecast-service/ — forecast service source + checkpoint/
03-watsonx-orchestrate/ — agent YAMLs, tools, connections
04-carbon-dashboard/ — Carbon dashboard source
bob-modes-skills/ — TTM Forecasting + FleetOps Agent Builder modes